

Lessons Learned from Leading Workshops about Geographically Distributed Agile Teams

Johanna Rothman and Shane Hastie

We've all read a lot about agile software development—the various flavors and methods sprouting up to tackle new issues in requirements, architecture, design, coding, and testing. But it's rare to hear real, tangible, objective lessons based on the experiences of those who have tried some form of agile development and failed. Get ready to take notes! —*Linda Rising, Associate Editor*

WE'VE BEEN TEACHING workshops about geographically distributed agile teams together and separately since March 2011. In that time, we've led workshops in New Zealand, Australia, Israel, Germany, and multiple times in the US, Saudi Arabia, and Canada. But it doesn't matter where we hold these workshops or where the project leaders or the company senior managers are located, we hear many of the same stories.

The players vary, the context is often different, but underneath it all, we've seen a set of common mistakes and success factors. In this article, we bring these common aspects together into a set of lessons learned, with some concrete advice on how to avoid the mistakes and leverage and amplify the successes.

The Lessons Learned

We separate our lessons learned into

project lessons (PLs), management lessons (MLs), and culture lessons (CLs).

PL#1: "We Thought Scrum Would Work"

Almost every project manager or lead who participated in our workshops started off by saying, "We thought Scrum would work." Scrum is a wonderful project management framework, but it's designed for a five- to seven-person, cross-functional, collocated team. Because our participants had geographically distributed projects, they immediately failed the collocation test.

However, more importantly, almost all of these teams were part of a larger effort: a program of work. A program is the coordination of several related projects to meet one larger business objective.¹ (Note that a program of projects is different from a computer program!) Because the program requires coordination and risk management across teams and within the organization, and Scrum by itself only provides Scrum of Scrums, the program didn't have sufficient coordination and risk management. Does that mean Scrum can't work? No—it just means that extending Scrum to a new-to-agile geographically distributed team is nontrivial.

Several scaled agile models offer some good advice in terms of going beyond the small, collocated team described as the agile "sweet spot."^{2,3} Finding the right scaling structure for your project and environment takes some experimentation, so use agile approaches to experiment and adapt to find what fits best for you.

PL#2: "We'll Force a Cross-Functional Team to Form"

One of the reasons Scrum didn't work was because senior management said, "Go do Scrum with these people from these silo'd teams. We'll give you testers

in India, developers in Germany, developers in Sweden, developers in Israel, product owners in Brazil, business analysts in North Carolina, and Scrum masters in California. We empower you to make yourselves a team.”

This is an example of just one of the geographically distributed teams we met. In this case, the developers were fairly close in time zones, but far away from their product owner and their business analyst. At least they were close to their tester—their one lonely and undereducated tester.

Even when colocated, it takes time and effort for a group of individuals to become a team, and this is harder and takes longer if they start out distributed. Put in the effort and time to allow people to get to know each other so they can truly jell.

We thought people knew Bruce Tuckman’s model of form/storm/norm/perform,⁴ which says that people need time to get to know each other to work together as a team, and that teams have stages that they proceed through. If you don’t take the time to form, you spend a lot of time in storming, and you never get to norming, never mind performing.

The team members we met knew the model, but the managers who created these distributed teams appeared to be unfamiliar with it or had forgotten that teams need that time to get to know each other.

Why did the managers forget? We suspect it’s because they work with each other all the time—that is, the managers continue to work with their peers on an ongoing basis. The managers’ teams don’t change, but the project teams do change, often out of sight of the managers who inflict the change.

The more time zones apart the team members are, the longer it takes to jell. Teams need interaction to be able to storm—to figure out their collaboration process, their ways of communicating, and their shared norms.

The harder it is to storm—the effect of many time zones—the longer it takes to get to norming. If you want to get to performing, it’s much easier if people travel to meet each other.

PL#3: Meeting Expense Is Cheaper than the Cost of Not Meeting

The teams that traveled to meet each other succeeded better than the teams that didn’t—in every single case.

We developed our workshop together as a geographically distributed pair, but this happened after meeting each other, establishing trust, and seeing how we each worked. We met first before deciding to work together remotely (Boston and New Zealand).

What astonished us was how many senior management teams flung people together, called them a team, and gave them no chance to establish trust before handing them multimillion-dollar efforts. When our workshop participants asked for travel budgets, they were told there was no money for travel.

It seems crazy to us that there’s no money to bring seven to 10 people together to break bread for a week, to learn how to work together to ensure their work. And in every case, the teams who spent the money and traveled at the beginning of the project to establish trust enjoyed a project with fewer misunderstandings and a product with fewer defects. The return on investment of the relatively small amount of money spent bringing the teams together could be measured in better collaboration, shortened decision-making cycles, fewer misunderstandings and rework, and higher-quality products with shorter time to market.

ML#1: Don’t Underpower Infrastructure in Remote Locations

We heard astonishing stories about how the remote teams didn’t have the same access to tools, sources, knowledge, and talent as the teams closer to head-

quarters. The closer the teams were to senior management, to headquarters, the more likely the teams were to be properly funded. This automatically creates an “us versus them” distance between the groups, and disadvantages the whole team.

With distributed teams, the mere ability to collaborate is determined by the tools available to the members with the lowest capability—it doesn’t help having fiber to the office in one location if another is restricted to a single dialup line shared by eight people. A daily standup over a Skype call that drops out every 40 seconds just frustrates everyone involved, and people quickly stop trying to communicate.

ML#2: Don’t Hire Testers 12 Hours Away to Save Money

Managers too often selected testers to be 12 hours away from developers because they didn’t understand that building software is a creative design activity. Unfortunately, some managers think that software is a manufacturing activity. As long as they think of software as construction, they’ll continue to believe that they can make cost-based decisions. In our case, the managers thought they were saving money on testing. In reality, they were costing the project time and money, confusing project cost and labor cost (see www.jrothman.com/blog/mpd/2010/03/wage-cost-and-project-labor-cost.html).

Distributing work to reduce cost is one of the worst reasons to form distributed teams. Examine why the team is being distributed and ensure that goals are actually achievable.

ML#3: Managers Can’t Keep Their Hands off People and on Projects

As soon as you have a geographically distributed team, it’s crucial to manage the project portfolio and have work flow through the team. Moving people

on and off disturbs the team and makes it difficult for its members to succeed at all. But many of the people we met had managers who were accustomed to picking people up and moving them around the organization. These managers weren't accustomed to managing the project portfolio.

The managers were now hit with a double whammy: they had to leave the team alone to complete project work either in flow or in an iteration, and they had to manage the project portfolio.⁵ These two challenges often drove managers into such chaos that they found it difficult to continue to support the transition to agile. So, just about the time the team hits its stride, the managers stop the transition because they couldn't take the required management transparency of committing to one project at a time and not moving people around like chess pieces. It's enough to make anyone go berserk.

CL#1: Different Isn't Wrong, but It Takes Effort to Overcome This Bias

People are different, and distributed teams often compound those differences because they take people who would otherwise most likely never come across one another and tell them, "You're a team—work together productively right now!"

We all bring our background, biases, experiences, history, and culture with us—these things make us who we are. Your home culture has hundreds of unwritten rules of behavior that "everyone knows," including social norms, unspoken hierarchies, and even simple things like which cutlery to use (or not use) depending on the meal being served.

Distributed team members don't have the same understanding of each other's culture. Our natural tendency is to assume that everyone has the same set of unwritten norms as we do. Of course, this is wrong, but that doesn't matter—unless team members

take the time to stop and think about how they communicate and what messages they convey through their behavior. If they don't make the effort to overcome this natural bias and think through their messages, especially their emails if they haven't met first, then misunderstanding and miscommunication are rife.

together on the project. This was the high point of the week, and everyone referred to it again and again during that project. You might not think knife washing has a cultural difference, but rest assured it can be.

You don't have to go as far as Johanna's team did, but take advantage of any opportunity to share a meal when

Moving people on and off disturbs the team and makes it difficult for its members to succeed at all.

A lot of research looks at these cultural dimensions⁶ and recommends spending time in a team talking about each other's cultures. Respectful curiosity goes a long way to creating understanding. Bringing people together and giving them the space to have these conversations is a great way to start a team building some shared norms.

CL#2: Sharing Food Is an Important Bonding Activity

One proven and powerful technique for building understanding in a team is to share food together, taking the time to understand the message of the meal.

When you bring people together, do it the way one of us (Johanna) did many years ago with a geographically distributed project team. She had the team work together during the day on "real" project work, and then one evening, she invited them to her home where they would all prepare dinner.

Everyone prepared their special food. The self-professed non-cooks brought flowers, seltzer, bread, beer, and wine; the rest of the team cooked up a storm. The kitchen was a disaster, and everyone ate too much, but they all learned a lot about working together in a too-small kitchen and how to work

you visit geographically distributed teammates. Sharing food will bond you in a way that a call or email will not.

CL#3: Create Team Norms around Risk and Being Done

When we taught the workshops, the participants often asked, "How do we help our project teams get to 'done' at the end of the iteration?" Another frequent question was, "How do we get to the end of the project?"

Collocated teams have these problems as well, but they can more easily conduct retrospectives and examine their behaviors and artifacts. It's not so easy for a geographically distributed team to do this. One of the most important ongoing discussions such a team needs to have is to define "done" for the story, the iteration, the release, the project, and the program, and then make sure the definition is understood and agreed on by all members of the team, as well as the management guiding them.

We believe in small stories that you finish in one- or two-week iterations that you then demo to your product owner or customer. That's how you manage "done" and project risk. You have release criteria for the project.¹



JOHANNA ROTHMAN, known as the “Pragmatic Manager,” helps organizational leaders see problems and risks in their product development. She has an MS in systems engineering from Boston University. Contact her at jr@jrothman.com.



SHANE HASTIE is chief knowledge engineer at Software Education, a training and consulting firm based in New Zealand. He has an MIM in information management from Victoria University of Wellington. Contact him at shaneh@softed.com.

You never let your work in progress grow, and you know where the project as a whole is headed. You keep heading on a course, checking in with the customer, but not every team believes in this approach.

So keep having the conversation, “What does done mean for us?” and “How much risk can we take?” If you keep discussing “done” and risk, you’ll make the issues obvious. You’ll also discover the cultural differences that often hide project risk until everything explodes.

Geographically Distributed Agile Isn’t an Oxymoron

You can make geographically distributed agile teams work, but you need to think about how to do it:

- Consider which agile approach you want to use. Do consider Scrum, but if you want to use it with a new-to-agile team, get training and use a coach. An even better idea for many teams is to use an unbranded form of agile that fits their specific context (see www.jrothman.com/

blog/mpd/2012/02/geographically-distributed-agile-teams-have-choices-for-their-lifecycles.html).

- Make sure you have feature teams, not silo’d teams. There are smart people all over the world, and you should take advantage of them, but taking one person from here and someone else from there and calling those people a team as if they were items from a menu doesn’t work.
- Outfit each team with the infrastructure it requires. Every team we teach this workshop to complains about this problem. Infrastructure can even mean training—don’t expect that a person who can spell “agile” can do agile.
- Beg, plead, whine; do whatever you need to do to get the project team or the core program team together at the beginning of the project/program to learn who each other is and how to work together. You will recoup that money and time very quickly.
- Most teams new to agile still have trouble with too-large stories. This occurs whether the team is geographically distributed or in the

same room. We recommend moving to a one-week iteration or using a Kanban board so the team can see and feel the work in progress and then eliminate work in progress.

These are the minimum conditions we’ve seen, not the only conditions you might need for your team. Your mileage will vary.

We’re considering adding a workshop for managers because many of the problems our participants encounter are management-generated. However, managers don’t feel the pain, so we have to find another way to reach them. Many potential participants have said, “We want this workshop as a webinar because we have no travel budget to attend a workshop that would help us fix our troubled projects!”

We’re learning how to do experiential training online. So far, we don’t know how, but we’re open to suggestions. ☺

References

1. J. Rothman, *Manage It! Your Guide to Modern, Pragmatic Project Management*, Dallas and Raleigh, 2007.
2. P. Kruchten, “The Frog and the Octopus: A Conceptual Model of Software Development,” July 2011; <http://arxiv.org/abs/1209.1327>.
3. J. Rothman, *Agile Program Management: Collaborating across the Organization*, Leanpub, to be published in 2013.
4. B.W. Tuckman and M.A.C. Jensen, “Stages of Small Group Development Revisited,” *Group and Organizational Studies*, vol. 2, 1977, pp. 419–427.
5. J. Rothman, *Manage Your Project Portfolio: Increase Your Capacity and Finish More Projects*, Dallas and Raleigh, 2009.
6. G. Hofstede et al., *Cultures and Organization: Software of the Mind*, McGraw Hill, 2010.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.